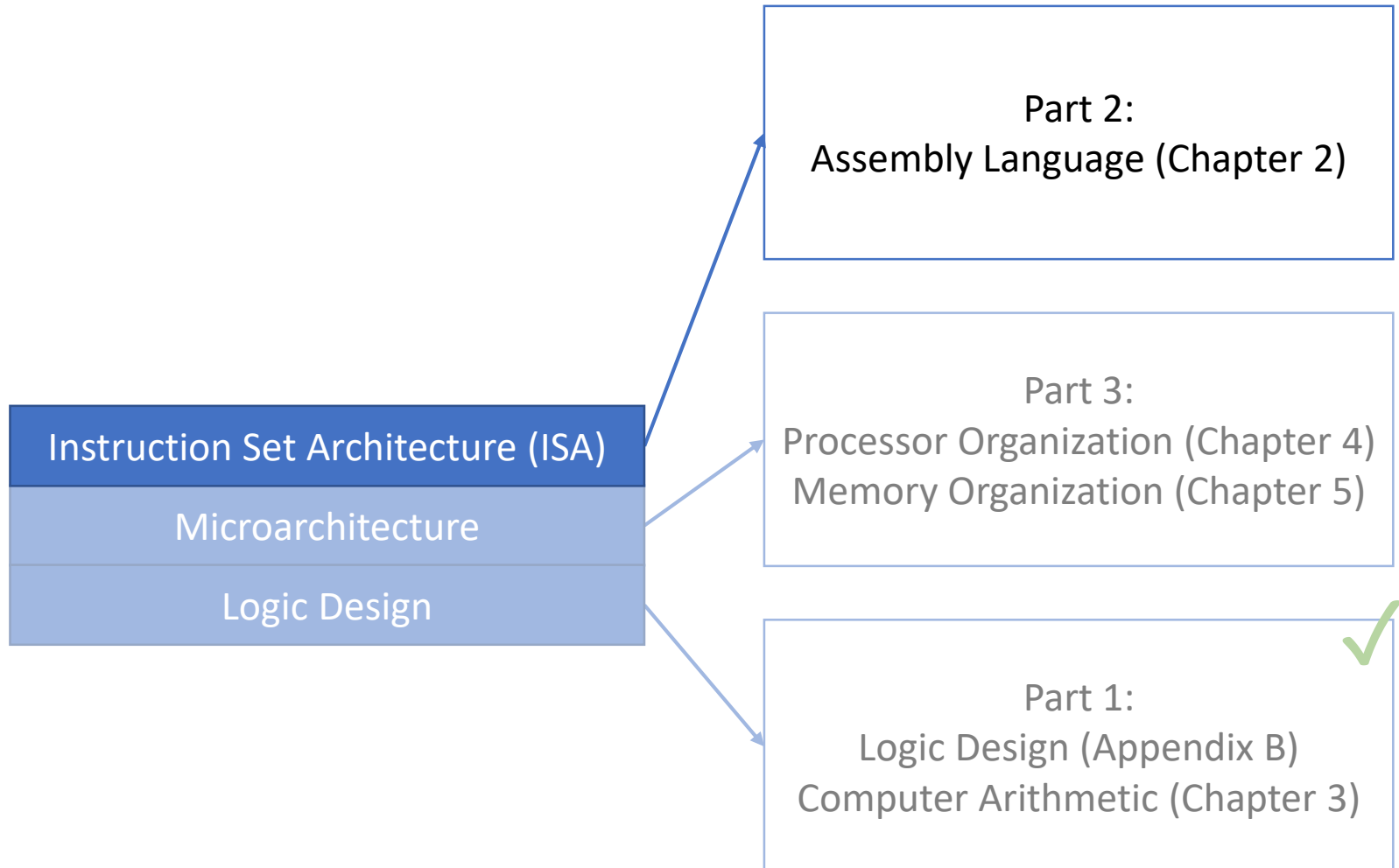


Lecture 15: Arithmetic and Logic Instructions

CMPS 221 – Computer Organization and Design

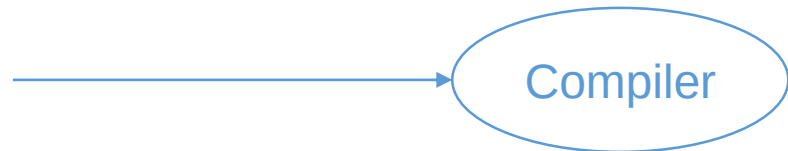
Part 2: Instruction Set Architecture



Assembly Language

```
swap (int v[], int k) {  
    int temp;  
    temp = v[k];  
    v[k] = v[k+1];  
    v[k+1] = temp;  
}
```

High-level Language Program
(what the programmer writes)



Compiler

```
swap:  slil $2, $5, 2  
       add $2, $4, $2  
       lw  $15, 0($2)  
       lw  $16, 4($2)  
       sw  $16, 0($2)  
       sw  $15, 4($2)  
       jr  $31
```

Assembly Language Program
(what the hardware understands)



Assembler

```
000000 00000 00101 0001000010000000  
000000 00100 00010 0001000000100000  
...
```

Machine (Object) Code
(what the hardware *really* understands)



Instruction Set

- An **instruction set** is the repertoire of instructions recognized by a computer
- Different computers have different instruction sets
 - e.g., x86, ARM, Power, RISC-V, MIPS

The MIPS Instruction Set

- The **MIPS** instruction set is used as the example throughout the book
- Stanford MIPS commercialized by MIPS Technologies (www.mips.com)
- Large share of embedded core market
 - Applications in consumer electronics, network/storage equipment, cameras, printers, ...
- Typical of many modern ISAs
- See MIPS Reference Data card in textbook

Arithmetic Instructions

- Add and subtract instructions
 - Three operands: two source, one destination
 $\text{add } a, b, c \quad \# \quad a = b + c$
- All arithmetic operations have this form

Arithmetic Example

- C code:

$$f = (g + h) - (i + j);$$

- Convert to sequence of three-operand instructions:

```
add t0, g, h    # temp t0 = g + h
add t1, i, j    # temp t1 = i + j
sub f, t0, t1   # f = t0 - t1
```

Register Operands

- Arithmetic instruction operands refer to registers stored in a register file
- MIPS has a 32×32 -bit register file
 - A 32-bit unit of data is called a **word**
 - Registers numbered 0 to 31
- Registers are used for frequently accessed data
 - Less frequently accessed data is placed in memory

Register Operand Names

- Registers have human-readable names that represent what they are typically used for
- Example names:
 - Registers 8-15 and 24-25 are named \$t0, \$t1, ..., \$t9
 - Typically used for “temporary” values
 - Registers 16-23 are named \$s0, \$s1, ..., \$s7
 - Typically used for “saved” variables
- More register names introduced later

Register Operand Example

- C code:

`f = (g + h) - (i + j);`

- `f, ..., j` in registers `$s0, ..., $s4`

- Compiled MIPS code:

`add $t0, $s1, $s2`

`add $t1, $s3, $s4`

`sub $s0, $t0, $t1`

The Constant Zero

- MIPS register 0 (named \$zero) always stores the constant 0
 - Cannot be overwritten
- Useful for common operations
 - e.g., move between registers (no need for special move instruction)
 `add $t2, $s1, $zero`
 - We'll see others later

Logical Operations

- Instructions for bitwise manipulation

Operation	C	Java	MIPS
Bitwise AND	&	&	and
Bitwise OR			or
Bitwise NOT	~	~	nor
Shift left	<<	<<	sll
Shift right	>>	>>>	srl

AND Operations

and \$t0, \$t1, \$t2

- Useful to mask bits in a word
 - Select some bits, clear others to 0

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0000 1100 0000 0000

OR Operations

or \$t0, \$t1, \$t2

- Useful to include bits in a word
 - Set some bits to 1, leave others unchanged

\$t2	0000 0000 0000 0000 0000 1101 1100 0000
\$t1	0000 0000 0000 0000 0011 1100 0000 0000
\$t0	0000 0000 0000 0000 0011 1101 1100 0000

NOT Operations

- Useful to invert bits in a word
 - Change 0 to 1, and 1 to 0

\$t1 0000 0000 0000 0000 0011 1100 0000 0000

\$t0 1111 1111 1111 1111 1100 0011 1111 1111

- MIPS has NOR 3-operand instruction
 - $a \text{ NOR } b = \text{NOT } (a \text{ OR } b)$

`nor $t0, $t1, $zero`

Shift Operations

`sll $t0, $s0, 3`

- Shift left logical
 - Shift left and fill with 0 bits
 - sll by i bits multiplies by 2^i

`srl $t0, $s0, 5`

- Shift right logical
 - Shift right and fill with 0 bits
 - srl by i bits divides by 2^i (unsigned only)

Immediate Operands

- An **immediate operand** is a constant value specified in an instruction
`addi $s3, $s3, 4`
- No subtract immediate instruction
 - Just use a negative constant
`addi $s2, $s1, -1`
- Limited to small constants up to 16 bits
 - We'll see why later

Immediate Operands Example

- C code:

$f = (g + 2) - (h - 3);$

- f, g, h in registers $\$s0, \$s1, \$s2$

- Compiled MIPS code:

`addi $t0, $s1, 2`

`addi $t1, $s2, -3`

`sub $s0, $t0, $t1`

32-bit Constants

- Most constants are small
 - 16-bit immediate is sufficient
- For the occasional 32-bit constant
 - Copies 16-bit constant to left 16 bits of rt
 - Clears right 16 bits of rt to 0
 - Combine with ori to create 32-bit constant

`lui $s0, 61`

0000 0000 0111 1101	0000 0000 0000 0000
---------------------	---------------------

`ori $s0, $s0, 2304`

0000 0000 0111 1101	0000 1001 0000 0000
---------------------	---------------------

Textbook Sections

- The content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 2.1, 2.2, 2.3, 2.6, 2.10